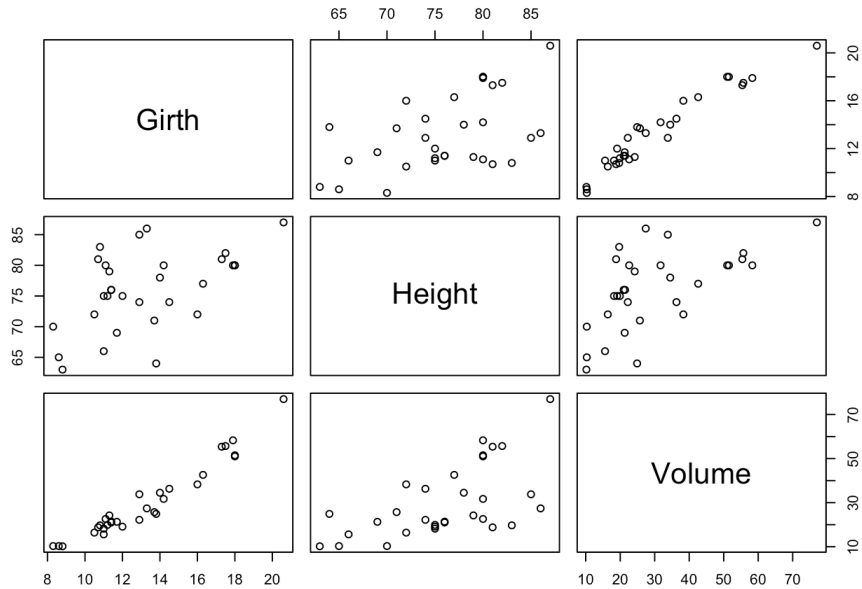
STAT 5703 - HW 4

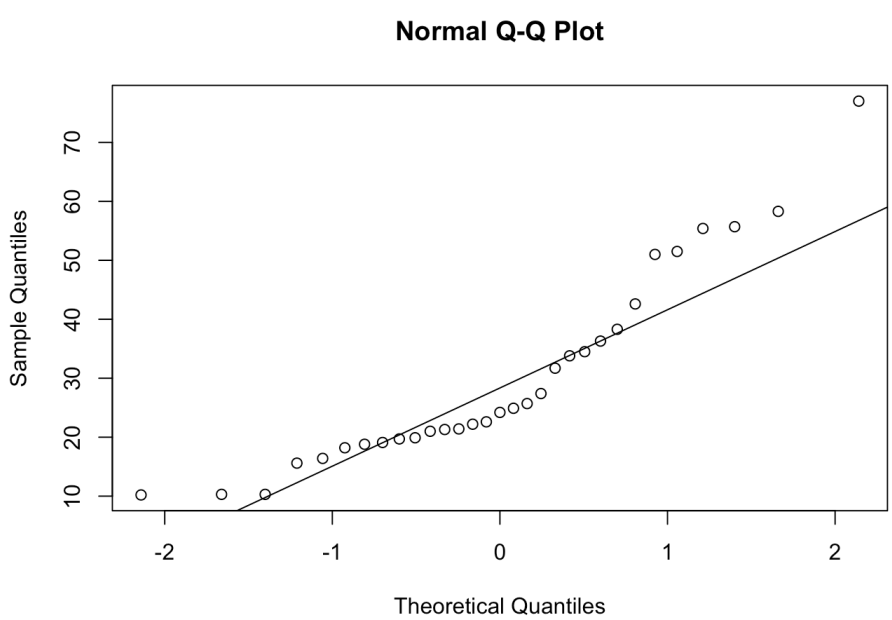
```r
# ====================================================================
# Setup
# Load the required library in a single command
# Load the data and assign it a sensible variable name
# ====================================================================
timber_data <- trees
timber_data
```

```
   Girth Height Volume
1    8.3     70   10.3
2    8.6     65   10.3
3    8.8     63   10.2
4   10.5     72   16.4
5   10.7     81   18.8
6   10.8     83   19.7
7   11.0     66   15.6
8   11.0     75   18.2
9   11.1     80   22.6
10  11.2     75   19.9
11  11.3     79   24.2
12  11.4     76   21.0
13  11.4     76   21.4
14  11.7     69   21.3
15  12.0     75   19.1
16  12.9     74   22.2
17  12.9     85   33.8
18  13.3     86   27.4
19  13.7     71   25.7
20  13.8     64   24.9
21  14.0     78   34.5
22  14.2     80   31.7
23  14.5     74   36.3
24  16.0     72   38.3
25  16.3     77   42.6
26  17.3     81   55.4
27  17.5     82   55.7
28  17.9     80   58.3
29  18.0     80   51.5
30  18.0     80   51.0
31  20.6     87   77.0
```

```r
# ====================================================================
# Part A
# Obtain and print the scatterplot matrix of all variables
# ====================================================================
pairs(timber_data)
```




```r
# ====================================================================
# Part B
# Check the normal distribution of the response variable Volume
# Generate a Q-Q plot and run a Shapiro-Wilk hypothesis test
# ====================================================================
qqnorm(timber_data$Volume)
qqline(timber_data$Volume)
```




```r
shapiro.test(timber_data$Volume)
```

```
    Shapiro-Wilk normality test

data:  timber_data$Volume
W = 0.88757, p-value = 0.003579
```

```r
# ====================================================================
# Part C
# Fit regular multiple regression model on log-transformed predictors
# Output the summary to check predictor significance
# ====================================================================
model_lm <- lm(Volume ~ log(Girth) + log(Height), data = timber_data)
summary(model_lm)
```

```
Call:
lm(formula = Volume ~ log(Girth) + log(Height), data = timber_data)

Residuals:
    Min      1Q  Median      3Q     Max 
-7.8190 -2.1355 -1.2785  0.9183 14.6848 

Coefficients:
            Estimate Std. Error t value Pr(>|t|)    
(Intercept) -234.888     53.925  -4.356 0.000161 ***
log(Girth)    61.269      5.058  12.114 1.19e-12 ***
log(Height)   25.045     13.784   1.817 0.079946 .  
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Residual standard error: 5.487 on 28 degrees of freedom
Multiple R-squared:  0.896, Adjusted R-squared:  0.8886 
F-statistic: 120.6 on 2 and 28 DF,  p-value: 1.734e-14
```

```r
# ====================================================================
# Part D
# Fit Gamma GLM with log link predicting Volume with intercept only
# Report the residual deviance
# ====================================================================
model_gamma_null <- glm(Volume ~ 1, family = Gamma(link = "log"), data = timber_data)
summary(model_gamma_null)$deviance
```

```
[1] 8.317201
```

```r
# ====================================================================
# Part E
# Fit Gamma GLM with log link predicting Volume by log(Girth)
# Report the residual deviance
# ====================================================================
model_gamma_girth <- glm(Volume ~ log(Girth), family = Gamma(link = "log"), data = timber_data)
summary(model_gamma_girth)$deviance
```

```
[1] 0.3840839
```

```r
# ====================================================================
# Part F
# Fit Gamma GLM with log link predicting Volume by log(Girth) and log(Height)
# Report the residual deviance
# ====================================================================
model_gamma_full <- glm(Volume ~ log(Girth) + log(Height), family = Gamma(link = "log"), data = timber_
summary(model_gamma_full)$deviance
```

```
[1] 0.1835153
```

```r
# ====================================================================
# Part G
# Approximate F-test comparing model from part E to part D
# The anova() function with test="F" automatically uses the Pearson
# estimate of dispersion from the larger model
# ====================================================================
anova(model_gamma_null, model_gamma_girth, test = "F")
```

```
Analysis of Deviance Table

Model 1: Volume ~ 1
Model 2: Volume ~ log(Girth)
```

```
  Resid. Df Resid. Dev Df Deviance      F    Pr(>F)
1        30     8.3172
2        29     0.3841  1   7.9331 593.72 < 2.2e-16 ***
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

```r
# ====================================================================
# Part H
# Approximate F-test comparing model from part F to part E
# Using Pearson estimate of dispersion from the larger model
# ====================================================================
anova(model_gamma_girth, model_gamma_full, test = "F")
```

```
Analysis of Deviance Table

Model 1: Volume ~ log(Girth)
Model 2: Volume ~ log(Girth) + log(Height)
  Resid. Df Resid. Dev Df Deviance      F    Pr(>F)
1        29    0.38408
2        28    0.18352  1  0.20057 31.206 5.604e-06 ***
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

```r
library(lme4)
library(dplyr)
library(ggplot2)

source("Recitation8.R")
```

```r
# ====================================================================
# Part A
# Fit random intercepts and random slopes model:
# Effect of ses.dev on mathach varies between schools
# ====================================================================
model_2a <- lmer(mathach ~ ses.dev + (ses.dev | school), data = df)

# Report the variance components
summary(model_2a)
```

```
Linear mixed model fit by REML ['lmerMod']
Formula: mathach ~ ses.dev + (ses.dev | school)
   Data: df

REML criterion at convergence: 46714.2

Scaled residuals:
     Min       1Q   Median       3Q      Max
-3.09680 -0.73193  0.01855  0.75386  2.89924

Random effects:
 Groups   Name        Variance Std.Dev. Corr
 school   (Intercept)  8.681   2.9464
          ses.dev      0.694   0.8331   0.02
 Residual             36.700   6.0581
Number of obs: 7185, groups:  school, 160

Fixed effects:
            Estimate Std. Error t value
(Intercept)  12.6362     0.2445   51.68
ses.dev       2.1932     0.1283   17.10

Correlation of Fixed Effects:
        (Intr)
ses.dev 0.009
```

```r
VarCorr(model_2a)
```

```
 Groups   Name        Std.Dev. Corr
 school   (Intercept) 2.94636
```

```
          ses.dev     0.83307  0.019
 Residual             6.05807
```

```r
# ====================================================================
# Part B
# Include contextual variables mean.ses and sector as fixed effects
# ====================================================================
model_2b <- lmer(mathach ~ ses.dev + mean.ses + sector + (ses.dev | school), data = df)

summary(model_2b)
```

```
Linear mixed model fit by REML ['lmerMod']
Formula: mathach ~ ses.dev + mean.ses + sector + (ses.dev | school)
   Data: df

REML criterion at convergence: 46543.3

Scaled residuals:
    Min      1Q  Median      3Q     Max
-3.1719 -0.7280  0.0124  0.7562  2.9307

Random effects:
 Groups   Name        Variance Std.Dev. Corr
 school   (Intercept)  2.3851  1.5444
          ses.dev      0.7004  0.8369   0.18
 Residual             36.7098  6.0589
Number of obs: 7185, groups:  school, 160

Fixed effects:
               Estimate Std. Error t value
(Intercept)     12.0634     0.1991  60.575
ses.dev          2.1950     0.1284  17.093
mean.ses         5.2463     0.3683  14.244
sectorCatholic   1.3722     0.3055   4.491

Correlation of Fixed Effects:
            (Intr) ses.dv men.ss
ses.dev      0.056
mean.ses     0.255 -0.004
sectorCthlc -0.698  0.002 -0.356
```

```r
# ====================================================================
# Part C
# Keep ses.dev slope fixed (random-intercept only model)
# ====================================================================
model_2c <- lmer(mathach ~ ses.dev + mean.ses + sector + (1 | school), data = df)

summary(model_2c)
```

```
Linear mixed model fit by REML ['lmerMod']
Formula: mathach ~ ses.dev + mean.ses + sector + (1 | school)
   Data: df

REML criterion at convergence: 46553.8

Scaled residuals:
    Min      1Q  Median      3Q     Max
-3.1831 -0.7285  0.0166  0.7519  2.9489

Random effects:
 Groups   Name        Variance Std.Dev.
 school   (Intercept)  2.369   1.539
 Residual             37.023   6.085
Number of obs: 7185, groups:  school, 160

Fixed effects:
               Estimate Std. Error t value
(Intercept)     12.1282     0.1992  60.885
ses.dev          2.1912     0.1087  20.163
mean.ses         5.3363     0.3689  14.464
sectorCatholic   1.2246     0.3061   4.001
```

```
Correlation of Fixed Effects:
            (Intr) ses.dv men.ss
ses.dev      0.000
mean.ses     0.256  0.000
sectorCthlc -0.699  0.000 -0.356
```

```r
fixef(model_2c)
```

```
   (Intercept)        ses.dev       mean.ses sectorCatholic 
     12.128212       2.191172       5.336261       1.224620 
```

```r
# ====================================================================
# Part D
# Compare Model 2c (reduced) with Model 2b (full)
# using a Likelihood Ratio Test
# ====================================================================
anova(model_2c, model_2b)
```

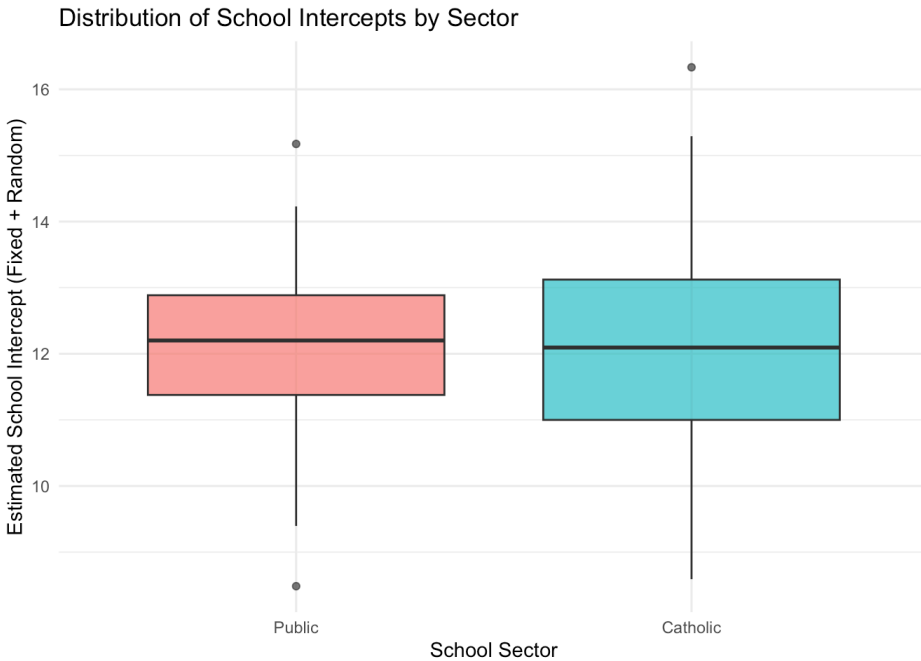
```
refitting model(s) with ML (instead of REML)

Data: df
Models:
model_2c: mathach ~ ses.dev + mean.ses + sector + (1 | school)
model_2b: mathach ~ ses.dev + mean.ses + sector + (ses.dev | school)
         npar   AIC   BIC logLik -2*log(L)  Chisq Df Pr(>Chisq)
model_2c    6 46560 46602 -23274     46548                     
model_2b    8 46554 46609 -23269     46538 10.176  2   0.006169 **
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

```r
# ====================================================================
# Part E
# Identify the "best 5" schools based on their intercepts
# (fixed intercept + random intercept)
# ====================================================================
# The coef() function automatically adds the fixed intercept and random
# effects together for each group.
school_intercepts <- coef(model_2c)$school

# Extract just the intercepts and keep school IDs
school_intercepts_df <- data.frame(
  school = rownames(school_intercepts),
  Total_Intercept = school_intercepts$`(Intercept)`
)

# Sort descending to find the top 5
top_5_schools <- school_intercepts_df %>%
  arrange(desc(Total_Intercept)) %>%
  head(5)

print(top_5_schools)
```

```
  school Total_Intercept
1   3427        16.33314
2   7688        15.29045
3   8628        15.24991
4   2655        15.17392
5   8193        14.93120
```

```r
# ====================================================================
# Part F
# Visualize random intercept distribution by sector with a boxplot
# ====================================================================
# First, we need to map the school sector back to our intercepts dataset
school_sectors <- df %>%
  select(school, sector) %>%
  distinct()

# Merge the total intercepts with the sector information
plot_data <- merge(school_intercepts_df, school_sectors, by = "school")
```

```r
# Create the side-by-side boxplot
ggplot(plot_data, aes(x = sector, y = Total_Intercept, fill = sector)) +
  geom_boxplot(alpha = 0.7) +
  theme_minimal() +
  labs(
    title = "Distribution of School Intercepts by Sector",
    x = "School Sector",
    y = "Estimated School Intercept (Fixed + Random)"
  ) +
  theme(legend.position = "none")
```




```r
# ====================================================================
# Problem 4 Setup
# Define global parameters and number of replications
# ====================================================================
R <- 100000       # Number of simulations
beta0 <- 10       # True intercept
beta1 <- 1        # True slope
n_vals <- c(25, 50, 100)
```

```r
# ====================================================================
# Part a) Data Generation & Average x
# Set seed, simulate x values once, and report averages
# ====================================================================
set.seed(549)

# Generate and store x values in a list so they remain fixed
x_list <- list()
for (n in n_vals) {
  x_list[[as.character(n)]] <- rnorm(n, mean = 0, sd = 1)
}

# Calculate and print the average of x for each sample size
x_averages <- sapply(x_list, mean)
print("Average x values for n = 25, 50, 100:")
```

```
[1] "Average x values for n = 25, 50, 100:"
```

```r
print(x_averages)
```

```
         25          50         100
 0.26090411 -0.15097751  0.05515922
```

```r
# ====================================================================
# Part b & c) Baseline Simulation (n=25, gamma=0)
# ====================================================================
n <- 25
gamma <- 0
x <- x_list[[as.character(n)]]

# 1. Calculate standard deviations (sqrt of variance)
sd_eps <- sqrt(exp(gamma * x))

# 2. Simulate errors (100,000 columns, 25 rows)
eps_mat <- matrix(rnorm(n * R, mean = 0, sd = sd_eps), nrow = n, ncol = R)

# 3. Compute responses Y
y_mat <- beta0 + beta1 * x + eps_mat

# 4. Fast Matrix OLS: beta_hat = (X'X)^(-1) X' Y
X_mat <- cbind(1, x)
H <- solve(t(X_mat) %*% X_mat) %*% t(X_mat)  # Projection matrix
beta_hats <- H %*% y_mat                     # Computes 100,000 regressions instantly

# Calculate and print averages
avg_beta0 <- mean(beta_hats[1, ])
avg_beta1 <- mean(beta_hats[2, ])

cat("Baseline Simulation (n=25, gamma=0)\n")
```

```
Baseline Simulation (n=25, gamma=0)
```

```r
cat("Average Beta_0:", avg_beta0, "(True = 10)\n")
```

```
Average Beta_0: 10.00032 (True = 10)
```

```r
cat("Average Beta_1:", avg_beta1, "(True = 1)\n")
```

```
Average Beta_1: 1.000137 (True = 1)
```

```r
# ====================================================================
# Part d) Expanded Heteroskedasticity Simulation
# Loop through all combinations of gamma and n
# ====================================================================
gamma_vals <- c(0, 1, 2)
results_het <- expand.grid(n = n_vals, gamma = gamma_vals)
results_het$avg_beta0 <- NA
results_het$avg_beta1 <- NA

for (i in 1:nrow(results_het)) {
  n <- results_het$n[i]
  g <- results_het$gamma[i]
  x <- x_list[[as.character(n)]]

  sd_eps <- sqrt(exp(g * x))
  eps_mat <- matrix(rnorm(n * R, mean = 0, sd = sd_eps), nrow = n, ncol = R)
  y_mat <- beta0 + beta1 * x + eps_mat

  X_mat <- cbind(1, x)
  H <- solve(t(X_mat) %*% X_mat) %*% t(X_mat)
  beta_hats <- H %*% y_mat

  results_het$avg_beta0[i] <- mean(beta_hats[1, ])
  results_het$avg_beta1[i] <- mean(beta_hats[2, ])
}

print("Heteroskedasticity Simulation Results:")
```

```
[1] "Heteroskedasticity Simulation Results:"
```

```r
print(results_het)
```

```
    n gamma avg_beta0 avg_beta1
1  25     0 10.000404 0.9995174
2  50     0 10.000585 1.0005788
3 100     0 10.000238 0.9999002
4  25     1  9.997789 0.9997980
5  50     1 10.000937 1.0016341
6 100     1  9.999652 1.0000319
7  25     2 10.001893 1.0123724
8  50     2 10.002826 1.0046889
9 100     2  9.998925 0.9986046
```

```r
# ====================================================================
# Part e) Autoregression Model Simulation
# Loop through all combinations of rho and n
# ====================================================================
rho_vals <- c(0, 0.5, 0.9)
results_ar <- expand.grid(n = n_vals, rho = rho_vals)
results_ar$avg_beta0 <- NA
results_ar$avg_beta1 <- NA

for (i in 1:nrow(results_ar)) {
  n <- results_ar$n[i]
  rho <- results_ar$rho[i]
  x <- x_list[[as.character(n)]]

  # Initialize error matrix
  eps_mat <- matrix(0, nrow = n, ncol = R)
  eps_mat[1, ] <- 3  # Set epsilon_1 = 3 for all 100,000 paths

  # Generate standard normal u_i
  u_mat <- matrix(rnorm((n - 1) * R, mean = 0, sd = 1), nrow = n - 1, ncol = R)

  # Build the autoregressive series iteratively
  for (j in 2:n) {
    eps_mat[j, ] <- rho * eps_mat[j - 1, ] + u_mat[j - 1, ]
  }

  y_mat <- beta0 + beta1 * x + eps_mat

  X_mat <- cbind(1, x)
  H <- solve(t(X_mat) %*% X_mat) %*% t(X_mat)
  beta_hats <- H %*% y_mat

  results_ar$avg_beta0[i] <- mean(beta_hats[1, ])
  results_ar$avg_beta1[i] <- mean(beta_hats[2, ])
}

print("Autoregression Simulation Results:")
```

```
[1] "Autoregression Simulation Results:"
```

```r
print(results_ar)
```

```
    n rho avg_beta0 avg_beta1
1  25 0.0  10.14391 0.9085485
2  50 0.0  10.06418 1.0264571
3 100 0.0  10.03240 0.9531898
4  25 0.5  10.28858 0.8141107
5  50 0.5  10.13401 1.0880337
6 100 0.5  10.06336 0.9250871
7  25 0.9  11.18104 0.7551705
8  50 0.9  10.62067 1.1688587
9 100 0.9  10.30271 0.9553665
```